

BHI3xx

Ultra-low power, high performance, programmable
Smart Sensor with integrated IMU

BHI3xx SDK Quick Start Guide

Document revision	2.1
Document release date	December 2025
Document number	BST-BHI3xx-AN000-07
Sales Part Number(s)	<u>BHI360: 0273017037; BHI380: 0273017039; BHI385: 0273017068</u>
Notes	Data and descriptions in this document are subject to change without notice. Product photos and pictures are for illustration purposes only and may differ from the real product appearance. The technical details and legal disclaimer of the respective product data sheet apply.

Table of Contents

1	Introduction to the SDK.....	6
2	Setup on Windows.....	7
2.1	Installing the compiler and support tools.....	7
2.2	Installing the SDK.....	7
2.3	Importing the SDK into Eclipse	8
3	Setup on Linux.....	12
3.1	Installing the ARC GNU toolchain and support tools	12
3.2	Installing the GNU toolchain.	13
3.3	Installing CMake and other dependencies.....	13
3.4	Installing the SDK on Linux.....	14
4	Importing the SDK into Visual Studio Code.....	14
4.1	Importing the SDK into VS Code	14
4.2	Building the SDK.....	15
4.3	Locate the Firmware	17
5	Building the SDK and loading firmware into the BHI360 (BHI380/BHI385).....	17
6	Adding a BSX based new custom virtual driver.....	18
6.1	Driver directory structure.....	18
6.2	Writing driver code	18
6.3	Adding the new sensor to down-sampling list.....	19
6.4	Selecting a driver ID.....	20
6.5	Driver CMakeLists.txt File	20
6.6	Brief introduction to the board configuration file	21
6.7	Modifying the board configuration file	22
6.8	Brief introduction to the SDK configuration file	23
6.9	Modifying the SDK configuration file	24
6.10	Building the custom firmware	24
6.11	Lean orientation example.....	25
7	Adding a non-Bosch Sensortec Fusion Library related new custom virtual driver	26

7.1	Driver directory structure.....	26
7.2	Writing driver code	26
7.3	Selecting a driver ID.....	28
7.4	Driver CMakeLists.txt file	28
7.5	Modifying the board configuration file	29
7.6	Modifying the SDK configuration file	29
7.7	Building the custom firmware	29
7.8	Altitude example.....	30
8	Integrating a library and applying it to the custom sensor driver	31
8.1	Library directory structure	31
8.2	Implementing a sensor driver that uses the library	31
8.3	Modifying the board configuration file	32
8.4	Modifying the SDK configuration file	32
8.5	Building the custom firmware	33
9	Glossary.....	34
9.1	Virtual Sensor.....	34
9.2	Driver ID	34
9.3	Sensor ID	34
10	Document history and modification	35

List of Figures

Figure 1: BHI360 SDK installer	7
Figure 2: Installation destination directory	8
Figure 3: Eclipse workspace prompt.....	9
Figure 4: Eclipse New Project Settings.....	10
Figure 5: Configuring the 'Builder Settings'	11
Figure 6: Configuring the build trigger's arguments.....	11
Figure 7: GNU toolchain releases download page	13
Figure 8: SDK structure in VS Code	15
Figure 9: Compiling on Windows	16
Figure 10: Architecture of physical and virtual drivers	18
Figure 11: Driver descriptor overview	19
Figure 12: Driver CMakeLists.txt.....	20
Figure 13: Board configuration file overview.....	21
Figure 14: Modifying the board configuration file.....	22
Figure 15: Overview of the SDK configuration file	23
Figure 16: Modifying the SDK configuration file.....	24
Figure 17: Driver descriptor overview	27
Figure 18: Driver CMakeLists.txt.....	28
Figure 19: Modifying the board configuration file.....	29
Figure 20: Modifying the SDK configuration file.....	29
Figure 21: Altitude output data structure.....	30
Figure 22: CMakeLists.txt	31
Figure 23: Modifying the board configuration file.....	32
Figure 24: Modifying the SDK configuration file.....	32

List of Tables

Table 1: Pre-build SDK directory structure on Windows.....	12
Table 2: Pre-build SDK directory structure on Linux.....	14
Table 3: Driver directory content.....	18
Table 4: sif and bus options	22
Table 5: Driver directory content.....	26
Table 6: Library directory content	31
Table 7: Driver directory content.....	31

Abbreviations

BST	Bosch Sensortec
BSX	Bosch Sensortec Fusion Library
FIFO	First In First Out
GCC	GNU Compiler Collection
RAM	Random Access Memory
SDK	Software Development Kit
USB	Universal Serial Bus
TRNG	True Random Number Generator
RDRAND	Read Random

1 Introduction to the SDK

This document briefly describes the process of developing firmware for the BHI360, BHI380 and BHI385. It provides instructions on how to

- set up the development environment
- build the SDK
- get started with custom configuration files.

For more details about hardware configuration, refer to *BHI360 Datasheet*, *BHI380 Datasheet* and *BHI385 Datasheet*. For more details about developing new drivers, refer to the following manual and user guide:

- *BHI3xx Programmer's Manual*
- *BHI3xx Evaluation Setup Guide*.

The BHI360 (BHI380/BHI385) SDK can be used to develop a custom firmware image. Customization includes:

- modifying the board configuration
- changing the mapping of pins
- changing the device orientation
- allocating memory to the FIFO
- creating custom drivers which can run algorithms or other tasks
- data injection for processor in the loop verification.

The BHI360 firmware built by using the BHI360 SDK can be downloaded to the BHI360's RAM.

The BHI380 firmware built by using the BHI380 SDK can be downloaded to the BHI380's RAM.

The BHI385 firmware built by using the BHI385 SDK can be downloaded to the BHI385's RAM.

For more details, refer to *BHI3xx Programmer's Manual*.

2 Setup on Windows

This chapter describes how to install the required tools on Windows. The BHI360 (BHI380/BHI385) SDK supports two toolchains: ARC GNU toolchain and Synopsys Metaware. This guide focuses on how to build the SDK with the ARC GNU toolchain. Since the SDK generates signed firmware images and the signing tool requires the True Random Number Generator feature of the CPU to generate a valid signature, the CPU used to build the SDK must support the RDRAND instruction.

2.1 Installing the compiler and support tools

The GNU Toolchain for ARC Processors can be downloaded from the [Synopsys Github Website](#). Download the file “arc_gnu_2022.09_ide_win_install.exe” or newer and run this setup installer executable. This will primarily install the Eclipse IDE and the ARC GNU Compiler.

Download the latest ninja tool from the [Ninja Release Website](#), extract ninja.exe from the ninja Windows package and replace the old one at the win64/bin folder under the BHI360 (BHI380/BHI385) SDK root path.

2.2 Installing the SDK

For Windows system, an SDK installer is provided. To install the SDK, do the following (take BHI360 SDK installation as an example):

1. Execute the *BHI360_SDK_Installer_V3.0.0.0.exe* or newer, accept the license agreement and click *Next*, as shown in Figure 1.

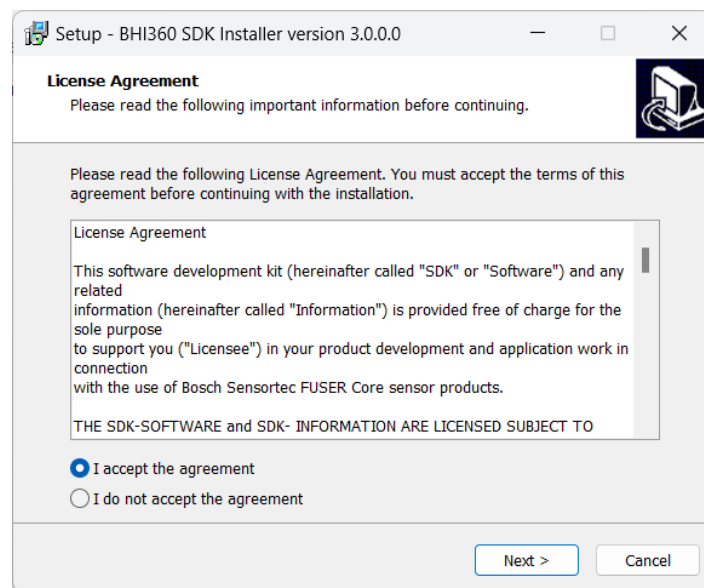


Figure 1: BHI360 SDK installer

2. Select the destination location for the SDK.

Then in the SDK destination directory, “*BHI360_SDK_VX. Y. Z*” is created.

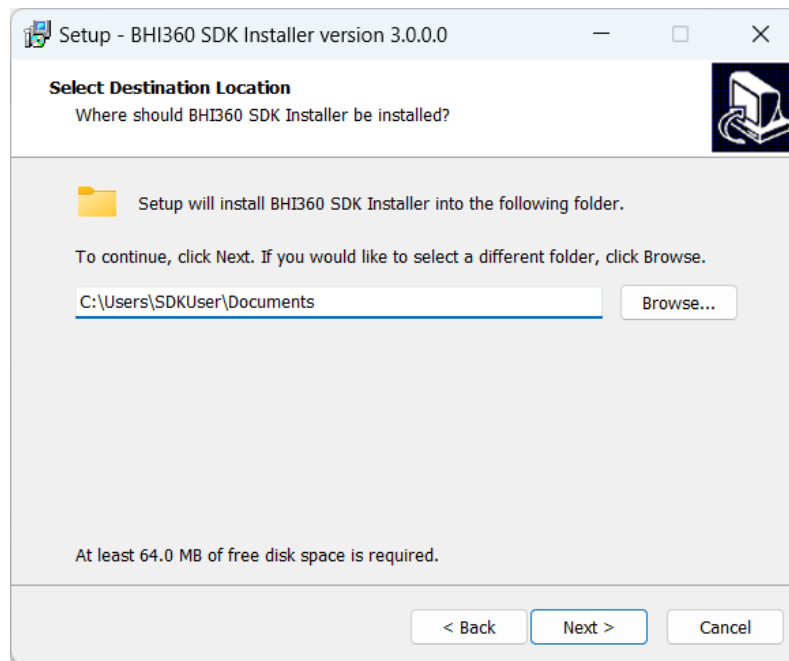


Figure 2: Installation destination directory

2.3 Importing the SDK into Eclipse

1. Set up Eclipse.
 - a. Run the Eclipse IDE by clicking on its shortcut on the Desktop, which should be generically named "ARC GNU IDE YYYY.MM(-rcN) Eclipse". For example, "ARC GNU IDE 2022.09 Eclipse".
 - b. During the first launch, you will be prompted to select a workspace. The default directory is an empty directory that stores multiple projects. You can select your preferred workspace directory.

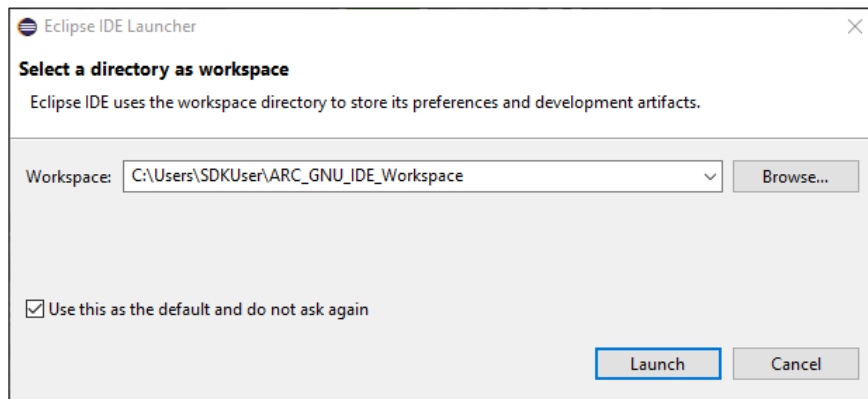


Figure 3: Eclipse workspace prompt

2. Import the BHI360 SDK as a project.
 - a. In the Eclipse IDE, go to *File > New > Makefile Project with Existing Code*.
 - b. In the prompt, type a project name, for example, as shown in the figure below. Select the SDK directory, choose `<none>` for Toolchain for Indexer Settings, and then click *Finish*.

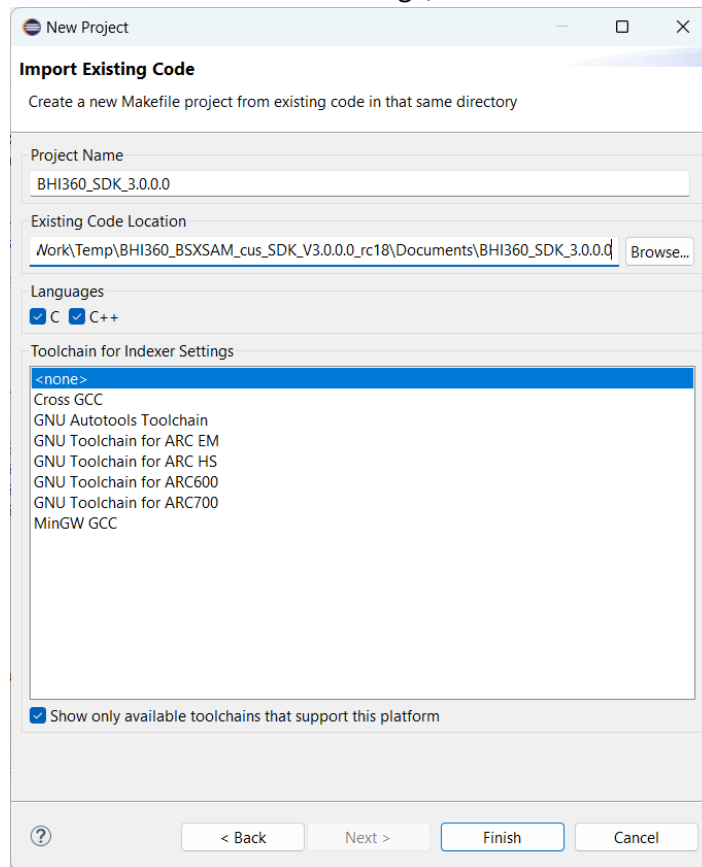


Figure 4: Eclipse New Project Settings

- c. If the Welcome tab is open, close it to reveal the *Project Explorer*.
3. Link the project build to the batch script that builds the firmware.
 - a. On Windows, the building of the firmware is managed by a batch script named *build.bat* which can be found in the root of the SDK directory with the specific building options `'-b imu -g USE_GCC'`.
 - b. Right-click on the *BHI360_SDK* project and select *Properties*.

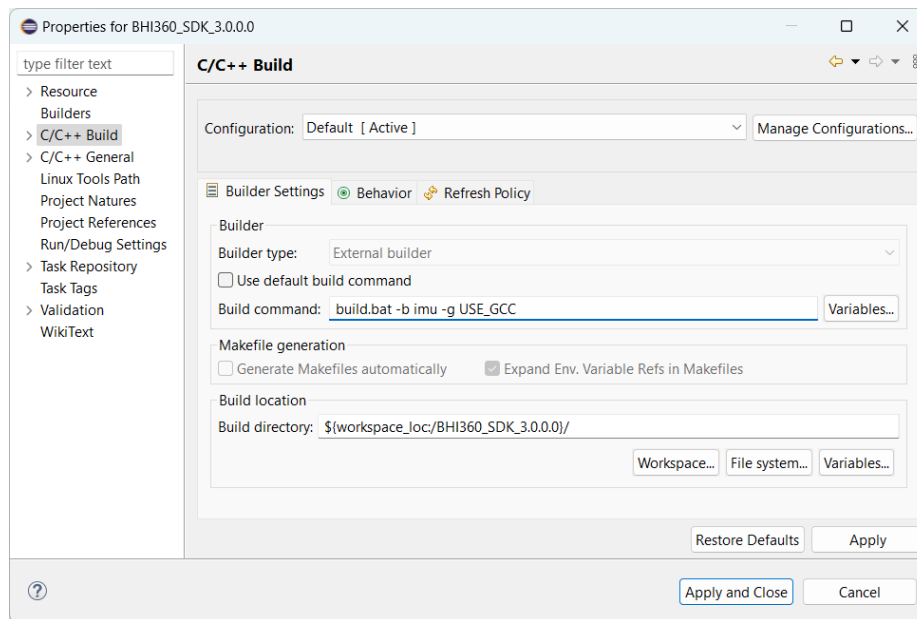


Figure 5: Configuring the 'Builder Settings'

- c. Under *C/C++ Build / Builder settings*, deselect *Use default build command* and refer to the image to select the build trigger. Click *Apply*.
- d. Under the *C/C++ Build / Behavior*, deselect *Clean* and remove the command *all* from the *Build behavior*, as shown in the figure below. Click *Apply and Close*.

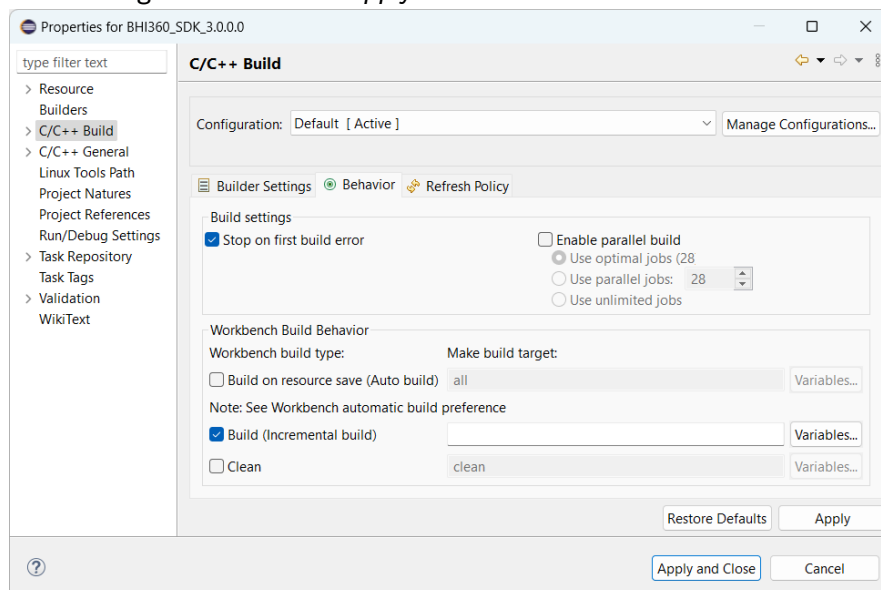


Figure 6: Configuring the build trigger's arguments

- e. Click on the build icon . This will run *build.bat* and the progress is visible in the console located at the bottom.

4. Locate the built firmware.

- a. The firmware build can be found under *release/gccfw* in the root directory of the SDK. If the firmware is built by Metaware rather than ARC GNU toolchain, it can be found under *release/fw* instead.

Table 1: Pre-build SDK directory structure on Windows

SDK File/Directory	Description
apps	Directory which contains the source code for applications running outside the sensor framework
boards	Configuration files for the supported development boards and sensors
cmake	CMake files used to build the SDK
common	Source code for initialization code and reference header files
docs	SDK documentation
drivers	Source codes of sensor drivers from Bosch Sensortec
drivers_custom	Source codes of additional custom drivers
gdb	Support files for using gdb
kernel	Exported symbols for kernel-mode firmware
libs	Linkable binary image and header files for API libraries
user	Entry code for user-mode firmware, source code for custom user-mode RAM patches
win64	Executable image manipulation utilities, command line interface
build.bat	Shell script used to set up build directory and build the specified target

3 Setup on Linux

3.1 Installing the ARC GNU toolchain and support tools

To get started, the following system requirements must be met:

- 64-bit Linux operating system (Ubuntu 14.04 LTS or later)
- At least 1.1 GB of free disk space.

Before the SDK can be used, ARC GNU toolchain, CMake, and other necessary dependencies must be installed.

The operations in this guide have been verified on Ubuntu 14.04 LTS and 16.04 LTS.

1. Download the ARC GNU toolchain.

The ARC GNU toolchain releases are available on the [Synopsys Github Website](#). A pre-built toolchain that supports elf32 little-endian hosts is required.

In this example, the 2022.09 release is used. This release can be downloaded from the same download page as the previous releases.

The right installation package to download is “arc_gnu_2022.09_prebuilt_elf32_le_linux_install.tar.gz”.

 arc_gnu_2022.09_ide_linux_install.tar.gz	1.56 GB
 arc_gnu_2022.09_ide_plugins.zip	871 KB
 arc_gnu_2022.09_ide_win_install.exe	872 MB
 arc_gnu_2022.09_prebuilt_arc32_uclibc_linux_install.tar.gz	76.2 MB
 arc_gnu_2022.09_prebuilt_arc32_uclibc_native_install.tar.gz	79.2 MB
 arc_gnu_2022.09_prebuilt_arc64_elf_linux_install.tar.gz	176 MB
 arc_gnu_2022.09_prebuilt_arc64_glibc_linux_install.tar.gz	108 MB
 arc_gnu_2022.09_prebuilt_arc64_glibc_native_install.tar.gz	111 MB
 arc_gnu_2022.09_prebuilt_elf32_be_linux_install.tar.gz	588 MB
 arc_gnu_2022.09_prebuilt_elf32_le_linux_install.tar.gz	562 MB
 arc_gnu_2022.09_prebuilt_glibc_be_archs_linux_install.tar.gz	120 MB
 arc_gnu_2022.09_prebuilt_glibc_le_archs_linux_install.tar.gz	119 MB
 arc_gnu_2022.09_prebuilt_glibc_le_archs_native_install.tar.gz	106 MB
 arc_gnu_2022.09_prebuilt_uclibc_be_arc700_linux_install.tar.gz	77.2 MB
 arc_gnu_2022.09_prebuilt_uclibc_be_archs_linux_install.tar.gz	89.7 MB
 arc_gnu_2022.09_prebuilt_uclibc_le_arc700_linux_install.tar.gz	76.2 MB
 arc_gnu_2022.09_prebuilt_uclibc_le_archs_linux_install.tar.gz	88.7 MB
 Source code (zip)	
 Source code (tar.gz)	

Figure 7: GNU toolchain releases download page

3.2 Installing the GNU toolchain.

- a. Run the following commands to extract the GNU toolchain installation package:

```
$ tar -xvf arc_gnu_2022.09_prebuilt_elf32_le_linux_install.tar.gz
$ sudo mkdir -p /opt/arc_gcc
$ sudo mv arc_gnu_2022.09_prebuilt_elf32_le_linux_install /opt/arc_gcc
```

- b. Run the following commands to verify the GNU toolchain has been installed successfully:

```
$ cd /opt/arc_gcc/arc_gnu_2022.09_prebuilt_elf32_le_linux_install/bin/
$ ./arc-elf32-gcc -dumpversion
```

- c. Update the PATH variable to include “opt/arc_gcc/arc_gnu_2022.09_prebuilt_elf32_le_linux_install/bin”. This can be done by modifying the shell start-up script as appropriate. For example, edit “/etc/profile” with the following command.

```
$ sudo nano /etc/profile
```

- d. Add the path to the file by adding the following line.

```
export PATH=$PATH:/opt/arc_gcc/arc_gnu_2022.09_prebuilt_elf32_le_linux_install/bin
```

3.3 Installing CMake and other dependencies

- a. To install CMake, run the following commands:

```
$ sudo apt-get install cmake
$ cmake --version
```

- b. To install the other dependencies or tools, if necessary, run the following commands.

```
$ sudo apt-get install libelf-dev
$ sudo apt-get install g++
$ sudo apt-get install lib32stdc++6
```

- c. It is highly recommended to install ninja to speed up the build process by parallel building.

```
$ sudo apt-get install ninja-build
```

3.4 Installing the SDK on Linux

The SDK is released as an installer “*BHI360_SDK_VX.Y.Z_Install.sh*”.

Take BHI360 SDK V3.0.0.0 as an example, to make the installer executable, run the following command:

```
$. /BHI360_SDK_V3.0.0.0_Install.sh
```

Bosch Sensortec License must be accepted by typing yes in the command line prompt. Then, the installer prompts to move to the preferred directory. The default installation directory is “***\$(HOME)/Bosch_Sensortec_Fuser2_SDK***”.

The SDK has the directory structure as shown in Table 2.

Table 2: Pre-build SDK directory structure on Linux

SDK File/Directory	Description
apps	Directory that contains the source code for applications running outside the sensor framework
boards	Configuration files for the supported development boards and sensors
cmake	CMake files used to build the SDK
common	Source code for initialization code and reference header files
docs	SDK documentation
drivers	Source codes of sensor drivers from Bosch Sensortec
drivers_custom	Source codes of additional custom drivers
gdb	Support files for using gdb
kernel	Exported symbols for kernel-mode firmware
libs	Linkable binary image and header files for API libraries
user	Entry code for user-mode firmware, source code for custom user-mode RAM patches
utils	Executable image manipulation utilities, command line interface
build.sh	Shell script used to set up a build directory and build the specified SDK

4 Importing the SDK into Visual Studio Code

To ensure the smooth import of the SDK into Visual Studio Code (VS Code), complete the following prerequisite preparations:

1. Download the VS Code installer from the official Visual Studio Code website: <https://code.visualstudio.com/>.
2. Run the installer to install the VS code on your computer.
3. Install extensions: C/C++, C/C++ Extension Pack, and CMake.

4.1 Importing the SDK into VS Code

1. Launch VS Code.
2. Click *File > Open Folder* and specify the SDK installation directory.

Take BHI360_SDK_3.0.0.0 on Windows as an example, the SDK structure is shown as below.

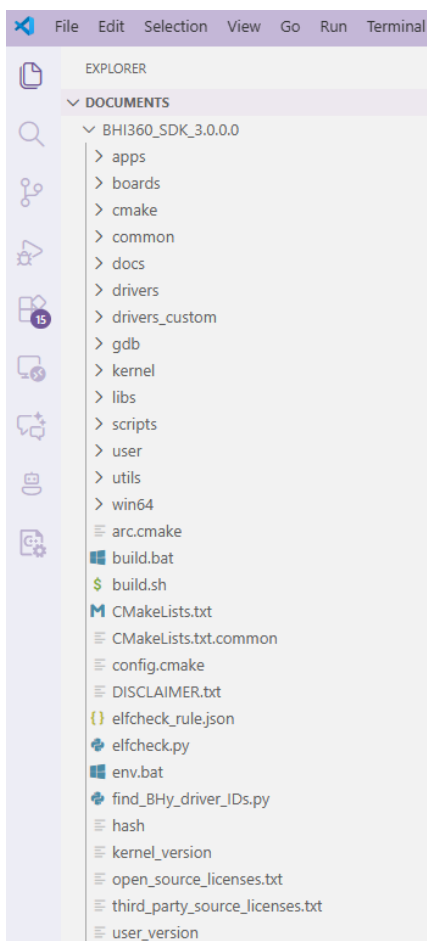


Figure 8: SDK structure in VS Code

The description of the SDK structure is provided in Table 2.

4.2 Building the SDK

1. Click *Terminal* > *New Terminal*.
2. Navigate to the root folder of BHI360 SDK, and check the SDK building options:

```

.\build.bat -h
SDK Version 3.0.0.0
Usage: .\build.bat [option]

Option:
  -b, --bsxsam          Select the bsxsam fusion.
                        Valid variables: imu, ndof, imulite
                        e.g.: -b imulite
  -a, --algo            Select the algorithm fusion
                        Valid variables: klio_cyclic
                        e.g.: -a klio_cyclic
  -g, --gcc             Select the compiler tool to GCC, otherwise use Metaware
                        Valid variables: USE_GCC
                        e.g.: -g USE_GCC
  -t, --target          Select the board config file
                        Valid variables: board config files in boards/
                        e.g.: -t Bosch_DataInject_BHI385_bsxsam_lite
  -h, --help            Show this help information
-----

Customer sdk building command tips:
--For normal release:
.\build.bat
--For ImuFusionX\NdoFusionX\ImuFusionXLite all in one SDK:
--Build ImuFusionX variant:
.\build.bat -b imu
--Build NdoFusionX variant:
.\build.bat -b ndof
--Build ImuFusionXLite variant:
.\build.bat -b imulite
--Build ImuFusionX\NdoFusionX\ImuFusionXLite with Klio variant:
.\build.bat -b imu -a klio_cyclic
--Build NdoFusionX variant with Klio variant:
.\build.bat -b ndof -a klio_cyclic
--Build ImuFusionXLite variant with Klio variant:
.\build.bat -b imulite -a klio_cyclic
-----

```

To build the SDK for the ImuFusionX variant, run the corresponding command:

- On Windows:

```
$ .\build.bat -b imu -g USE_GCC
```

- On Linux:

```
$ ./build.sh -b imu -g USE_GCC
```

```

PS C:\Work\Temp\BHI360_BSXSAM_cus_SDK_V3.0.0.0_rc18\Documents\BHI360_SDK_3.0.0.0> .\build.bat -b imu -g USE_GCC
SDK Version 3.0.0.0
Using cmake binary: C:\Work\Temp\BHI360_BSXSAM_cus_SDK_V3.0.0.0_rc18\Documents\BHI360_SDK_3.0.0.0\win64\cmake\bin\cmake.exe
Using cpack binary: C:\Work\Temp\BHI360_BSXSAM_cus_SDK_V3.0.0.0_rc18\Documents\BHI360_SDK_3.0.0.0\win64\cmake\bin\cpack.exe
Using ninja binary: C:\Work\Temp\BHI360_BSXSAM_cus_SDK_V3.0.0.0_rc18\Documents\BHI360_SDK_3.0.0.0\win64\bin\ninja.exe
----Running cmake generator
-- The C compiler identification is GNU 11.2.0
-- The ASM compiler identification is GNU
-- Found assembler: C:/arc_gnu/bin/arc-elf32-gcc.exe

```

Figure 9: Compiling on Windows

4.3 Locate the Firmware

For the BHI360, the image for RAM is generated. With successive build triggers, all previously generated files under the *build* and *release* directories are removed and new firmware files are generated under the ***release/gccfw*** or ***release/fw*** folder.

The generated “*.fw” file can be verified by using the bhy2cli tool. The bhy2cli is a command line tool based on the COINES tool that interfaces with the BHI360(BHI380/BHI385) through Bosch Sensortec’s application board. The tool can be used to load and run standard and custom firmware images among other features.

For example, running the command

```
$ bhy2cli -b release\fw\Bosch_Shuttle3_BHI360_bsxsam.fw -c 37:25
```

loads the firmware file `Bosch_Shuttle3_BHI360_bsxsam.fw` for the board configuration `Bosch_Shuttle3_BHI360_bsxsam.cfg` and switches on streaming of sensor ID 37 at 25Hz to the terminal. Refer to *BHI360-BHI380 Evaluation Setup Guide* for more information on building the *bhy2cli* tool.

5 Building the SDK and loading firmware into the BHI360 (BHI380/BHI385)

On Windows, clicking on the build icon  in the Eclipse IDE or executing the build.bat script with the command “build.bat -b imu” will trigger the build process.

On Linux, run the build script in its root:

```
$ ./build.sh -b imu
```

build and *release* directories are created after the build script is executed. If both the ARC GNU compiler and the Metaware compiler are available on the path, the Metaware compiler is used. To override this behavior and force the use of the ARC GNU compiler, add the option “-g USE_GCC” as an argument to the build script. If only the GNU compiler exists, the option “-g USE_GCC” is not necessary.

```
$ ./build.sh -b imu -g USE_GCC
```

```
$ ./build.bat -b imu -g USE_GCC
```

For the BHI360 (BHI380/BHI385), one image for RAM is generated. With successive build triggers, all previously generated files under the *build* and *release* directories are removed and new firmware files are generated under the *release/gccfw* or *release/fw* folder.

The generated “*.fw” file can be verified by using the bhy2cli tool. The bhy2cli is a command line tool based on the COINES tool that interfaces with the BHI360 (BHI380/BHI385) through Bosch Sensortec’s application board. The tool can be used to load and run standard and custom firmware images among other features.

For example, running

```
$ bhy2cli -b release\gccfw\Bosch_Shuttle3_BHI360_bsxsam.fw -c 37:25
```

loads the firmware file `Bosch_Shuttle3_BHI360_bsxsam.fw` for the board configuration `Bosch_Shuttle3_BHI360_bsxsam.cfg` and switches on streaming of the [sensor ID](#) 37 at 25 Hz to the terminal. Refer to *BHI3xx Evaluation Setup Guide* for more information on using the *bhy2cli* tool.

6 Adding a BSX based new custom virtual driver

In order to demonstrate how one can add a custom driver to the SDK, two drivers, *VirtBSXLeanDeviceOrientation* and *VirtBSXCustomCorrectedAccelDataSource*, have already been included in the SDK as examples but have not been used in any of the firmware images.

Both *VirtBSXLeanDeviceOrientation* and *VirtBSXCustomCorrectedAccelDataSource* are in the *drivers_custom* directory of the SDK. *VirtBSXCustomAccelDataSource* receives accelerometer data from the Bosch Sensortec Fusion Library but does not send it to the host. Instead, it triggers *VirtBSXLeanDeviceOrientation* which receives the data, processes it, and stores the processed data in the requested FIFO.

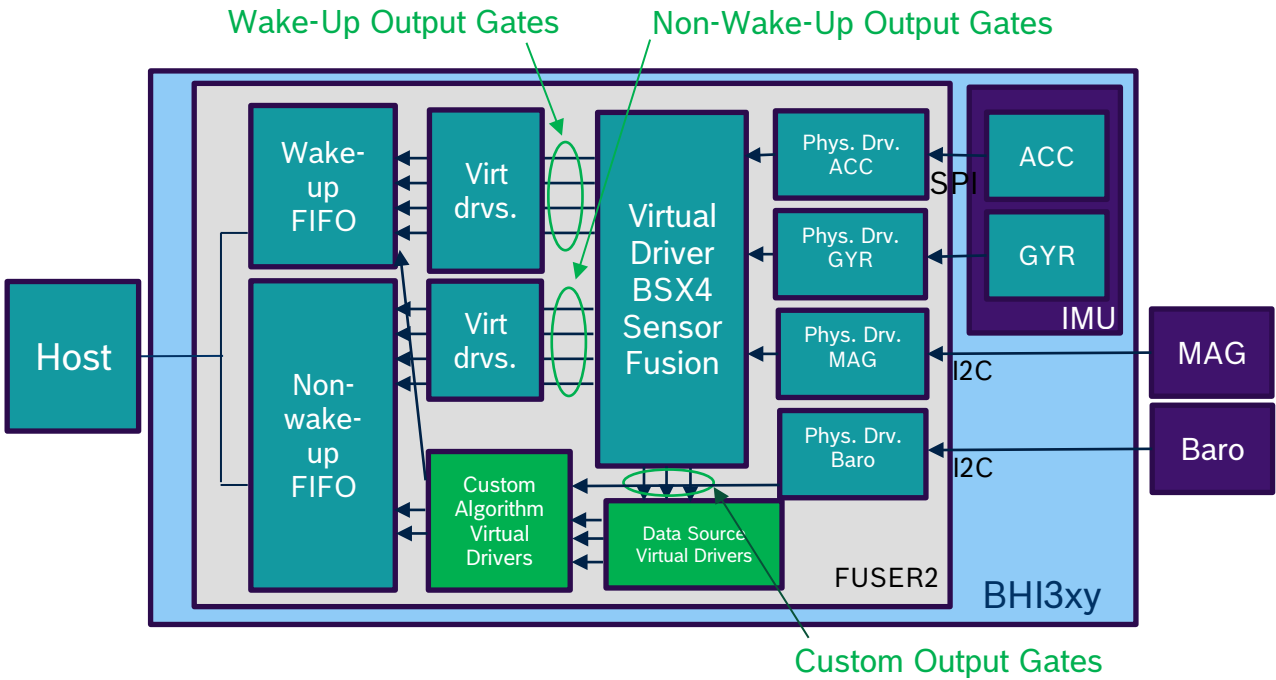


Figure 10: Architecture of physical and virtual drivers

For more information on how to develop a new physical sensor driver or [virtual sensor](#) driver in the SDK, refer to *BHI3xx Programmer's Manual*.

6.1 Driver directory structure

The sensor driver code must be in its own directory under the *drivers_custom* directory of the SDK. The directory name should indicate the device name and driver type, for example, *VirtBSXLeanDeviceOrientation*.

Table 3: Driver directory content

File in Driver Directory	Description
CMakeLists.txt	Build description of the driver
VirtBSXLeanDeviceOrientation.c	Source code of the driver
Header file	Header file typically defining register locations and other constants for the driver if needed

6.2 Writing driver code

The dependency between the two [virtual sensors](#) is described below.

“hidden = TRUE” means the sensor is not visible to host, and it only provides data source.

VirtBSXCustomCorrectedAccelDataSource is the data source of *VirtBSXLeanDeviceOrientation*.

```
VIRTUAL_SENSOR_DESCRIPTOR VirtualSensorDescriptor
descriptor_virt_bsx_custom_corrected_accel_data_source
= {
    .physicalSource = {
        .sensor = {
            .type = {
                .value = BSX_INPUT_ID_ACCELERATION,
                .flags = DRIVER_TYPE_PHYSICAL_FLAG,
            },
        },
    },

    .info = {
        .id = DRIVER_ID,
        .version = DRIVER_REV,
    },

    .type = {
        .value =
        SENSOR_TYPE_BSX(BSX_CUSTOM_ID_ACCELERATION_CORRECTED),
        .flags = DRIVER_TYPE_VIRTUAL_FLAG,
        .wakeup_ap = FALSE,
        .hidden = TRUE,
    },
    .expansionData = {
        .f32 = OUTPUT_SCALING_FACTOR,
    },

    .maxRate = 800.0F,
    .minRate = 1.5625F,

    .outputPacketSize = sizeof(output_3axis_t),
    .priority = PRIORITY_2,

    .initialize = NULL,
    .handle_sensor_data = bsx_custom_data_collector,
};
```

```
VIRTUAL_SENSOR_DESCRIPTOR VirtualSensorDescriptor
descriptor_virt_bsx_lean_device_orientation = {
#ifdef USE_BSXSAM
    .triggerSource = {
        .sensor = {
            .type = {
                .value =
                SENSOR_TYPE_BSX(BSX_CUSTOM_ID_ACCELERATION_CORRECTED)
            ,
                .flags = DRIVER_TYPE_VIRTUAL_FLAG,
            },
        },
    },
#endif
    .info = {
        .id = DRIVER_ID,
        .version = DRIVER_REV,
    },

    .type = {
        .value =
        SENSOR_TYPE_BSX_LEAN_DEVICE_ORIENTATION
    ,
        .flags = DRIVER_TYPE_VIRTUAL_FLAG,
        .wakeup_ap = FALSE,

        .maxRate = 800.0F,
        .minRate = 1.5625F,

        .outputPacketSize = sizeof(output_t),
        .priority = PRIORITY_2,

        .initialize = ldo_initialize,
        .handle_sensor_data = ldo_handle_sensor_data,
        .mode_changed = ldo_on_power_mode_changed,
    };
```

The [Sensor ID](#) is made visible to the host.

Figure 11: Driver descriptor overview

For more detailed and complete information on how to program sensor drivers, refer to Chapter 4.8.4 of *BHI3xx Programmer's Manual*.

6.3 Adding the new sensor to down-sampling list

Add the new sensor, which relies on the BSXSAM (ImufusionXLite\ImufusionX\NdoofusionX) outputs, to *user/RamPatches/custom_virtual_sensor_support_bsxsam.c*. More details can be referred in Chapter 4.8.4 of *BHI3xx Programmer's Manual*.

6.4 Selecting a driver ID

To add a new [virtual sensor](#) driver, the first step is to select the available driver ID for compilation. Unless the driver to be developed is already included in the SDK, you may choose any unused 8-bit number. There is a python script under the root directory of the SDK. Running it will show the existing driver names and associated driver IDs. Using this script will need an existing installation of [Python](#).

```
$ python find_BHy_driver_IDs.py
```

In this excerpt, the driver IDs **131** and **132** in the Driver CMakeLists.txt file (See section 6.5) are selected. Each driver has a unique driver ID.

6.5 Driver CMakeLists.txt File

The *CMakeLists.txt* mentioned below file automatically pulls in the sources from each driver. It is used by the build system at link time to associate the driver ID listed with a driver's object file. More driver IDs can be defined in the same way. Usually, you do not need to modify it.

Take *drivers_custom/VirtBSXLeanDeviceOrientation/CMakeLists.txt* for example:

```
SET(DRIVER_ID 121)

get_filename_component( DRIVER_KEY ${CMAKE_CURRENT_LIST_DIR} NAME)

project(${DRIVER_KEY} C)

FILE(GLOB SOURCES "*.c")

IF(BUILDING_SDK)
    include_directories(../../libs/BSXSupport/includes/
                        ../../libs/BSX/includes/
                        ../../user/RamPatches/)
ELSE()
    include_directories(../../../../libs/BSXSupport/includes/
                        ../../../../libs/BSX/includes/
                        ../../../../user/RamPatches/)
ENDIF()

ADD_ARC_DRIVER(${DRIVER_KEY} ${DRIVER_ID} ${SOURCES})
```

Figure 12: Driver CMakeLists.txt

6.6 Brief introduction to the board configuration file

Board configuration files are used to specify the configuration for a firmware build. A board configuration file consists of a global configuration section, a physical driver configuration section and a virtual driver configuration section. Lines can be commented with a hash (#) and are commented until the end of the current line.

- Default configuration of GPIO pins
- Sensor interface configurations (SPI, I2C masters)
- Allocation of FIFO memory
- CPU speed: long run (20MHz) or turbo (50MHz)
- Building firmware for Host boot
- Configuration parameters for BSX fusion library
- List of physical drivers to be linked into the firmware file and their characteristics
- List of virtual drivers to be linked into the firmware file

All board configuration files are in the *boards* directory.

Take `boards/Bosch_Shuttle3_BHI360_BMM150.cfg` as an example:

[illegible]

Figure 13: Board configuration file overview

Table 4: sif and bus options

sif	M1	M2	M3
0	SPI0	SPI1	I2C1
1	SPI0	I2C0	I2C1
2	I2C0	SPI1	I2C1

For more details about the sif configuration, refer to BHI360/BHI380/BHI385 Datasheet. For details about the board configuration file, refer to *BHI3xx Programmer's Manual*.

6.7 Modifying the board configuration file

To add the *VirtBSXLeanDeviceOrientation* and *VirtBSXCustomCorrectedAccelDataSource* virtual sensors to the *Bosch_Shuttle3_BHI360_bsxsam_LDO.fw*, you must add a new configuration file *boards/Bosch_Shuttle3_BHI360_bsxsam_LDO.cfg* (take *Bosch_Shuttle3_BHI360_bsxsam.cfg* as the reference) and add the virtual drivers to the virtual sensor list in the respective “*.cfg” file as shown below.

```
...
#Virtual Drivers,maxRate
131, 400.000000 # VirtImuFusionCustomCorrectedAccelDataSource: IMU fusion Corrected Accel data
source
...
121, -1.000000 # VirtBSXLeanDeviceOrientation: LDO
...

8, 400.000000 # VirtImuFusionCore: IMU fusion Core
252, 400.000000 # VirtImuFusionGameRV: IMU fusion GameRV
200, 400.000000 # VirtImuFusionWakeupGameRV: IMU fusion Wakeup GameRV
203, 400.000000 # VirtImuFusionRawAccel: IMU fusion Raw Accel
204, 400.000000 # VirtImuFusionWakeupRawAccel: IMU fusion Wakeup Raw Accel
...
```

Link *VirtBSXCustomCorrectedAccelDataSource* (Driver ID: 131) and *VirtBSXLeanDeviceOrientation* (Driver ID: 121) into the *Bosch_Shuttle3_BHI360_bsxsam_LDO.fw*

Figure 14: Modifying the board configuration file

6.8 Brief introduction to the SDK configuration file

In brief, all SDK generated firmware images include both the pre-built kernel image and user images. This configuration file includes board configuration files, enabled drivers, libraries, etc.

The SDK has one configuration file *common/config.7189_di03_rtos_bhi360_BSXSAM.cmake*, which can be edited as needed.

<pre>... set(IMU_BOARDS Bosch_Shuttle3_BHI360_bsxsam Bosch_Shuttle3_BHI360_bsxsam_turbo Bosch_Shuttle3_BHI360_HeadOrientation ...) ...</pre>	The BOARDS variable which includes IMU_BOARDS/IMULITE_BOARDS/NDOF BOARDS/KLIO_CYCLIC_BOARDS describes which of the target boards' configurations are to be built. When the "build.sh" or "build.bat" script is executed, only the firmware for those specific boards
<pre>set(DRIVERS_NO_SOURCE BHI360Accel BMM150Mag ... VirtHangDetection)</pre>	The DRIVERS_NO_SOURCE variable describes which drivers (including physical and virtual drivers) are already present as library files in the SDK.
<pre>set(ENABLED_DRIVERS VirtImuFusionRawAccel ... #Example Injection driver AccelInject ... \${DRIVERS_NO_SOURCE})</pre>	Drivers whose sources need to be built, such as custom drivers, should be directly added to the ENABLED_DRIVERS variable.
<pre>... set(RAM_PATCHES getBuildTime ...)</pre>	Source and header files of ram patches that need to be built should be directly added to the RAM_PATCHES and RAM_PATCHES_HEADERS variables.
<pre>set(RAM_PATCHES_HEADERS custom_virtual_sensor_support_bsxsam ...)</pre>	

Figure 15: Overview of the SDK configuration file

6.9 Modifying the SDK configuration file

To build a firmware that contains the reference custom drivers for the target board configuration, *common/config.7189_di03_rtos_bhi360_BSXSAM.cmake* needs to be modified as shown below.

6.10 Building the custom firmware

To build the IMU boards, the name 'imu' of the bsxsam fusion can be passed as an argument. For example, on Linux, this would look like,

<pre> set(IMU_BOARDS Bosch_Shuttle3_BHI360_bsxsam Bosch_Shuttle3_BHI360_bsxsam_turbo ... Bosch_Shuttle3_BHI360_bsxsam_LDO) ... set(ENABLED_DRIVERS VirtBSXLeanDeviceOrientation VirtImuFusionCustomCorrectedAccelDataSource #Example Injection driver AccelInject \${DRIVERS_NO_SOURCE}) ... # ram patches set(RAM_PATCHES getBuildTime ... custom_virtual_sensor_support_bsxsam bsxsam_virtual_sensor_support ...) # headers to be included set(RAM_PATCHES_HEADERS ... custom_virtual_sensor_support_bsxsam bsxsam_virtual_sensor_support ...) </pre>	Add <i>Bosch_Shuttle3_BHI360_bsxsam_LDO</i> to the IMU_BOARDS variable. Add <i>VirtBSXLeanDeviceOrientation</i> and <i>VirtImuFusionCustomCorrectedAccelDataSource</i> to the ENABLED_DRIVERS variable. Make sure <i>custom_virtual_sensor_support_bsxsam</i> is added to the RAM_PATCHES and RAM_PATCHES_HEADERS variables.
--	--

Figure 16: Modifying the SDK configuration file

```
$ ./build.sh -b imu
```

The build.bat file for Windows can accept similar arguments.

```
$ ./build.bat -b imu
```

The IMU firmware are now available under *release/gccfw* as *Bosch_Shuttle3_BHI360_bsxsam_LDO.fw*. Like with the reference firmware, you can use the bhy2cli like below to load the firmware and view the lean orientation sensor's output using the generic type of handle.

```
$ bhy2cli -b release/gccfw/Bosch_Shuttle3_BHI360_bsxsam_LDO.fw -a 160:"Lean
Orientation":2:c:c -c 160:1
```


6.11 Lean orientation example

With the new custom virtual drivers inside the firmware, the host should also be able to configure the sensor ID and parse sensor events from the FIFO.

The corresponding host side example of the virtual sensor `VirtBSXLeanDeviceOrientation` called “Lean Orientation” is provided separately. Refer to BHI360-BHI380/5 Evaluation Setup Guide for more information on how to verify and evaluate the aforementioned newly integrated virtual sensors.

7 Adding a non-Bosch Sensortec Fusion Library related new custom virtual driver

This chapter takes *VirtAltitude* for example to describe how to add a non-Bosch Sensortec Fusion Library related new custom virtual sensor driver. The steps are the same as in Chapter 5, except that some details vary depending on the actual requirements.

VirtAltitude is in the *drivers_custom* directory. It creates custom altitude data and sends it to the respective FIFO.

For more information on how to develop a physical/virtual sensor driver in the SDK, refer to *BHI3xx Programmer’s Manual*.

7.1 Driver directory structure

The sensor driver code must be in its own directory in the *drivers_custom* directory of the SDK. The directory name should reflect the device name and driver type, for example *VirtAltitude*.

Table 5: Driver directory content

File in Driver Directory	Description
CMakeLists.txt	Build description of the driver
VirtAltitude.c	Source code of the driver
Header file	Header file typically defining register locations and other constants for the driver if needed

7.2 Writing driver code

Below is code snippets *VirtAltitude* driver in Figure 17, which explains its trigger source and how to exchange reference sea level values with the host through the parameter interface.

For more information on how to program sensor drivers, refer to *BHI3xx Programmer’s Manual*.

```

...
#define DRIVER_REV          (4u)
...

#define PARAM_PAGE_OPTIONAL_SDK          (8)
#define OPTIONAL_SDK_PARAM_ALTITUDE_SEE_LEVEL    (0x00)

bool optional_sdk_page_write_handler(uint8_t param, uint16_t length, uint8_t buffer[])
...

bool optional_sdk_page_read_handler(uint8_t param, uint16_t length, uint8_t buffer[], uint16_t *ret_length)
...

static SensorStatus virt_altitude_initialize_sensor(VirtualSensorDescriptor *self)
{
    (void) registerWriteParamHandler(PARAM_PAGE_OPTIONAL_SDK, optional_sdk_page_write_handler);
    (void) registerReadParamHandler(PARAM_PAGE_OPTIONAL_SDK, optional_sdk_page_read_handler);
    verbose("altitude initialize\n");
    return SensorOK;
}
...

VIRTUAL_SENSOR_DESCRIPTOR VirtualSensorDescriptor virt_altitude_descriptor =
{
    .triggerSource =
    {
        .sensor =
        {
            .type =
            {
                .value =
                    SENSOR_TYPE_INPUT_BMP_PRESSURE
                ,
            },
            .flags = DRIVER_TYPE_PHYSICAL_FLAG,
        },
    },
    .physicalSource =
    {
        .sensor =
        {
            .type =
            {
                .value = SENSOR_TYPE_INPUT_BMP_PRESSURE
            }
        }
    }
}

```

The driver's version number is 4.

Use the Parameter page 0x08, index 0x00 to set and get the reference pressure at sea level. It is 4 bytes register to contain an unsigned 32 bit value. Host can set and get this

VirtAltitude's trigger source is physical pressure sensor.

VirtAltitude's sensor ID is a visible ID, which means that it is visible to the HOST.

Figure 17: Driver descriptor overview

7.3 Selecting a driver ID

To add a new virtual sensor driver, the first step is to select the available virtual driver ID for compilation. Unless the driver to be developed is already included in the SDK, you may choose any unused 8-bit number.

In this excerpt, the driver ID **123** in the `CMakeLists.txt` file is selected. Each driver has a unique driver ID.

7.4 Driver `CMakeLists.txt` file

The `CMakeLists.txt` file pulls in all the sources from the root directory of each driver. It is used by the build system while linking to associate the driver ID listed with a driver's object file. Additional driver IDs can be defined in the same way. This generic file typically needs no modification.

Take `drivers_custom/VirtAltitude/CMakeLists.txt` for example:

```
SET(DRIVER_ID 123)
get_filename_component( DRIVER_KEY ${CMAKE_CURRENT_LIST_DIR} NAME)

project(${DRIVER_KEY} C)

FILE(GLOB SOURCES "*.c")

ADD_ARC_DRIVER(${DRIVER_KEY} ${DRIVER_ID} ${SOURCES})
```

Figure 18: Driver `CMakeLists.txt`

7.5 Modifying the board configuration file

The example below describes how to include *VirtAltitude* in *Bosch_Shuttle3_BHI360_BMP58X_bsxsam.fw* by editing the existing configuration file “*\$SDK/boards/Bosch_Shuttle3_BHI360_BMP58X_bsxsam.cfg*” as shown below.

```
...
#Virtual Drivers,maxRate
223, -1.000000 # VirtBMPPressure: pressure depends on a physical pressure source.
229, -1.000000 # VirtWakeupBMPPressure: wakeup pressure depends on a physical pressure source.
211, -1.000000 # VirtBMPTemperature: temperature depends on a physical temp source.
212, -1.000000 # VirtWakeupBMPTemperature: wakeup temperature depends on a physical temp
source.
123, 16.000000 # VirtAltitude: depends on a physical pressure source.
```

Link *VirtAltitude* (Driver ID: 123) into the *Bosch_Shuttle3_BHI360_BMM150_BMP58X_bsxsam.fw*

Figure 19: Modifying the board configuration file

7.6 Modifying the SDK configuration file

In order to build a firmware that contains the *VirtAltitude* virtual driver for the target board configuration the *common/config.7189_di03_rtos_bhi360_BSXSAM.cmake* needs to be modified as shown below.

```
set(IMU_BOARDS
    Bosch_Shuttle3_BHI360_bsxsam
    Bosch_Shuttle3_BHI360_bsxsam_turbo
    Bosch_Shuttle3_BHI360_bsxsam_HeadOrientation
    ...
    Bosch_Shuttle3_BHI360_BMP58X_bsxsam
)
...
set(ENABLED_DRIVERS
    VirtImuFusionRawAccel
    VirtImuFusionRawGyro
    ...
    VirtAltitude
    ...
    ${DRIVERS_NO_SOURCE}
)
```

Add the configure file
*Bosch_Shuttle3_BHI360_BM
P58X_bsxsam* to the
IMU_BOARDS variable.

Add *VirtAltitude* to the
ENABLED_DRIVERS variable.

Figure 20: Modifying the SDK configuration file

7.7 Building the custom firmware

As described in Chapter 2 for Windows and Chapter 3 for Linux system, trigger the respective build.

The custom firmware is now available in *release/gccfw* as *Bosch_Shuttle3_BHI360_BMP58X_bsxsam.fw*. Like with the reference firmware, you can run the *bhy2cli* like below to load the firmware and view the lean orientation sensor's output using the generic type of handle.

```
$ bhy2cli -a 161:"Altitude":4:s32 -b release\fw\Bosch_Shuttle3_BHI360_BMP58x_bsxsam.fw -c
161:1
```

7.8 Altitude example

With new virtual sensor drivers in the firmware, a new sensor data parser should also be added to the host. An example of the virtual sensor VirtAltitude is provided separately.

The virtual altitude sensor's output unit is in centimeters.

For information on how to verify and evaluate the BHI360's virtual sensors, refer to *BHI3xx Evaluation Setup Guide*.

```
typedef struct {  
    Sint32  altitude;  
} __attribute__((packed)) output_t;
```

Here defines VirtAltitude output data as 4 bytes, so HOST side should parse altitude data into 4 bytes.

Figure 21: Altitude output data structure

8 Integrating a library and applying it to the custom sensor driver

This chapter describes how to integrate a library and use it in a driver. The library can be found under *libs/template* and the driver under *drivers_custom/VirtIntegrateLibTemplate*. For information on the development of a custom driver, refer to Chapter 6 and 7.

8.1 Library directory structure

The library directory and its files must be in the *libs* directory of the SDK. The directory name should indicate the library name, for example, *libs/template*. Table 6 lists the contents of the library directory.

Table 6: Library directory content

Source in Library Directory	Description
CMakeLists.txt	Build description of the library
template.sdk.cmake	CMake file of the library
libtemplate.a	Library file
includes	Header files directory of the library

8.2 Implementing a sensor driver that uses the library

For implementing a sensor driver, the contents of the driver directory are shown in Table 7.

Table 7: Driver directory content

Source in Library Directory	Description
CMakeLists.txt	Build description of the driver
VirtIntegrateLibTemplate.c	Driver file

Content of CMakeLists.txt. Note the includes required.

```
SET(DRIVER_ID 164)

get_filename_component( DRIVER_KEY ${CMAKE_CURRENT_LIST_DIR} NAME)

project(${DRIVER_KEY} C)

FILE(GLOB SOURCES "*.c")

IF(BUILDING_SDK)
include_directories(../../libs/template/includes/)
ELSE()
include_directories(../../libs/template/includes/)
ENDIF()
add_definitions("-DDESCRIPTOR_NAME=virt_${DRIVER_KEY}_desc")

ADD_ARC_DRIVER(${DRIVER_KEY} ${DRIVER_ID} ${SOURCES})
```

Figure 22: CMakeLists.txt

8.3 Modifying the board configuration file

To build a firmware that contains library and driver, the board configuration file needs modification.

Using the *boards/Bosch_Shuttle3_BHI360_bsxsam.cfg* as reference.

```
#Virtual Drivers,maxRate
164, -1.000000 #VirtIntegrateLibTemplate: an example for integrate library
8, 400.000000 # VirtImuFusionCore: IMU fusion Core
252, 400.000000 # VirtImuFusionGameRV: IMU fusion GameRV
200, 400.000000 # VirtImuFusionWakeupGameRV: IMU fusion Wakeup GameRV
```

Figure 23: Modifying the board configuration file

8.4 Modifying the SDK configuration file

To build a firmware that contains library and driver, the SDK configuration file needs modification.

SDK configuration file *common/config.7189_di03_rtos_bhi360_BSXSAM.cmake*

```
.....
SET(BUILD_LIBS
  ${EXPORT_LIB_SOURCE}
  ${EXPORT_LIB_BINARIES}
  DMA
  SensorInterfaceInit
  SensorInterfaceRAM
  DMAUnitTests
  HIDUnitTests
  HostInterfaceStreamingRAM

  BSXSupport
  OscTrim
  SensorCalibration
  Outerloop
  template
)

.....
# libraries linked to standard board images
set(BOARDS_LIBS
  MetawareDouble
  MetawarePrintf
  Template
  ...
)
...

set(ENABLED_DRIVERS
```

Figure 24: Modifying the SDK configuration file

8.5 Building the custom firmware

As described in Chapter 2 for Windows and Chapter 3 for Linux system, trigger the respective build. Then you can test the driver in the same way as described in Chapter 6.

9 Glossary

9.1 Virtual Sensor

A Virtual Sensor is a term used to identify the output of one or more algorithms. This output is available in the FIFO and can be identified and referenced by the host of the BHI360(BHI380/BHI385) using a Sensor ID.

9.2 Driver ID

A virtual sensor driver is responsible for implementing the interface between the Software Framework and the algorithm, among other tasks. Each driver has a unique ID in the SDK which is referred to as the Driver ID. This Driver ID is used to select which driver can be included into a firmware build.

9.3 Sensor ID

The Sensor ID is a unique identifier for a Virtual sensor. This Sensor ID is defined as an 8-bit unsigned integer value. A list of all Virtual sensors and their corresponding sensor IDs also known as a FIFO event IDs are described in the datasheet in the table Overview of FIFO Event IDs. The Sensor ID is defined as part of the driver's descriptor.

10 Document history and modification

Rev. No	Chapter	Description of modification/changes	Date
1.0	All	Main release	2020-02-04
1.1	All	Updated references	2020-02-24
1.2	All	Added Glossary	2020-04-24
1.3	All	Updated bhy2cli command formats	2020-05-18
1.4	5.9	Updated bhy2cli command to enable lean orientation	2021-10-20
1.5	2.1; 2.3; 3.1	Update compiler version number	2023-04-18
1.6	5.11;6.8	Review the examples	2024-05-10
1.7	2.3	Update contents and pictures	2024-08-24
1.8	All	Add BHI385 contents	2024-12-31
1.9	5, 6, 7	Update contents	2025-04-15
2.0	All	Update contents	2025-05-28
2.1	All	Update contents for all-in-one SDK	2025-12-24

Bosch Sensortec GmbH

Gerhard-Kindler-Straße 9
72770 Reutlingen / Germany

contact@bosch-sensortec.com
www.bosch-sensortec.com

Modifications reserved

Document number: BST-BHI3xx-AN000-07